COMP5318 Assignment 2: Image Classification

Group:123 550394172 550378183 550894694

1 Introduction

Classifying tissue images is a useful task for studying automated support for histopathology. Image patches can share similar colours while differing in texture and tissue structure, making it important to examine which information a classifier uses. Reliable performance also needs to be considered alongside computational cost and errors on individual classes. Comparing classifiers helps assess whether the performance gained from learning image features directly justifies the additional computation, or whether a model using fixed features offers a useful alternative when resources are limited.

This study compares Random Forest (RF), a multilayer perceptron (MLP), and a convolutional neural network (CNN) on the assignment-provided nine-class PathMNIST subset. These methods offer different ways of learning from images: RF and MLP use extracted features, while CNN learns spatial patterns from pixels. We examine how representation and hyperparameter choices affect performance, then compare the selected models using test accuracy, macro F1, class-level errors, and runtime. The aim is to identify the strengths and limitations of each approach on this dataset, rather than to establish clinical suitability.

2 Data

2.1 Data Description and Exploration

PathMNIST is a nine-class colon histopathology benchmark included in MedMNIST v2 [1]. We used the assignment-provided subset and splits [5], containing 32,000 training images and 8,000 test images. Each image contains 28×28 RGB pixels, stored as unsigned 8-bit integers with values from 0 to 255. We retained the supplied numeric labels from 0 to 8. These sample counts refer to the assignment data rather than the original benchmark splits.

Table 1 shows the training class distribution. Class 8 is the largest and class 6 the smallest, with a count ratio of approximately 1.72. We therefore report macro F1 alongside accuracy to give each class equal weight when assessing performance.

Table 1: Class distribution in the supplied training set of 32,000 images.

Class	0	1	2	3	4	5	6	7	8
Training count	3,490	3,431	3,505	3,656	2,950	4,290	2,728	3,253	4,697

Figure 1 shows one training image from each class, together with its local RGB mean and standard-deviation maps. The mean maps preserve broad colour patterns, while the standard-deviation maps show variation within each block. These summaries form part of the 357-dimensional feature representation used by the selected RF and MLP models. In the displayed examples, the background patch is relatively uniform, whereas tissue patches vary in local texture despite sharing similar pink and purple colours. This suggests that colour alone may be insufficient for classification, although one example per class cannot represent all within-class variation.

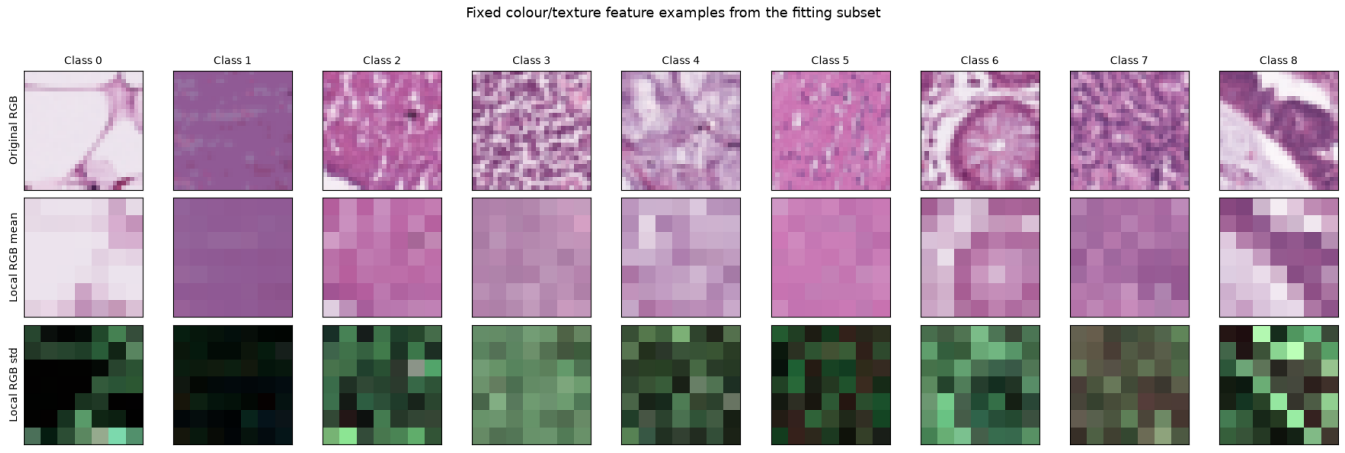


Figure 1: Training examples from classes 0–8, with one example per class from the fitting subset. Rows show the original RGB images, local RGB means, and local RGB standard deviations. Each summary map contains 7×7 blocks, calculated from non-overlapping 4×4 pixel regions. Standard-deviation maps share a common display scale; their colours represent channel-wise variability, not the original tissue colours.

All three models used the same stratified 80/20 split of the supplied training set, with random seed 42. This gave 25,600 fitting examples and 6,400 validation examples. Sharing the split kept the validation samples consistent across the RF, MLP, and CNN experiments. After model selection, each model was retrained on all 32,000 training examples and evaluated on the supplied test set.

2.2 Pre-processing

2.2.1 Image Representations and Feature Extraction

For the RF experiments, pixel intensities were divided by 255 before constructing three candidate representations. Flattened RGB images provided 2,352 features. Average pooling over non-overlapping 2×2 regions reduced this to 588 features, preserving broad colour patterns at the cost of some spatial detail.

The third representation combined local and global colour statistics. Each image was divided into 49 non-overlapping 4×4 blocks. The RGB means and standard deviations of these blocks contributed $49 \times 3 \times 2 = 294$ features. Normalised 16-bin histograms for each colour channel added 48 features, and five quantiles per channel (10%, 25%, 50%, 75%, and 90%) added 15. This produced the 357-dimensional representation used by the selected RF and the MLP.

Block statistics retain coarse spatial information and local variation, while histograms and quantiles describe colour distributions without depending on exact pixel positions. These summaries reduce dimensionality but cannot preserve all tissue structure. Their parameters were fixed, and each image was transformed independently, without fitting statistics across validation or test samples.

RF did not require feature standardisation: dividing pixel values by 255 provided a consistent range for feature extraction rather than a requirement for tree-based learning. No PCA, augmentation, or class reweighting was used in the RF pipeline. Pooling and the compact descriptor provided alternatives for reducing the input dimensionality.

2.2.2 MLP Feature Standardisation

The MLP used the shared 357-dimensional colour and spatial feature vectors. We standardised each feature using the mean and standard deviation of the fitting subset, then applied the same transformation to validation data. This placed features on comparable scales for gradient-based optimisation, following the input normalisation principles discussed by Goodfellow et al. [6, Ch. 8]. Fitting the scaler on the fitting subset avoided using validation information to estimate these statistics.

After model selection, the scaler was refitted on all 32,000 training examples and used to transform the 8,000 test examples. Missing-value imputation was unnecessary because the feature arrays contained no missing or non-finite values.

2.2.3 CNN Image Preprocessing

The CNN used the original $28 \times 28 \times 3$ RGB images rather than the extracted feature vectors. Pixel values were converted to floating point and divided by 255. Each colour channel was then standardised by subtracting its mean and dividing by its standard deviation, computed over the fitting subset.

The same channel statistics were applied to the validation images. For final training, the statistics were recomputed using all 32,000 training images and then applied unchanged to the test images. Validation and test images therefore did not contribute to fitting the preprocessing transformations.

Channel standardisation centres and scales the inputs, which can improve the conditioning of gradient-based optimisation [6, Ch. 8]. It preserves the full pixel grid for convolutional learning, whereas the fixed descriptor retains only coarse spatial summaries. We did not run a controlled comparison of preprocessing methods, so we do not attribute the CNN's performance gain to standardisation alone.

We retained the supplied resolution and did not crop the images, since cropping the already small 28×28 patches would discard tissue information. We did not add PCA: the RF and MLP already used a compact 357-dimensional representation, while the CNN required spatially arranged inputs. Rotation and flip augmentation could introduce useful variation, but we kept augmentation outside the current search to focus on the three selected CNN hyperparameters. Its potential benefit remains a question for future experiments.

3 Methods

3.1 Theory

3.1.1 Random Forest

We selected Random Forest (RF), covered in Week 5B [2], as the traditional classifier. It can capture nonlinear relationships among colour and spatial features without requiring a neural network. This provides a useful comparison with MLP on the same feature representation and with CNN learning directly from images.

RF combines decision trees trained on bootstrap samples drawn with replacement from the training data. At each node, a random subset of features is considered, and a split is chosen to reduce Gini impurity. Sampling both examples and features encourages diversity among trees, allowing their combined predictions to reduce variance compared with a single tree [3]. In the scikit-learn implementation used here, class probabilities are averaged across trees, and the class with the highest mean probability is predicted [4].

3.1.2 Fully Connected Neural Network

A multilayer perceptron (MLP) combines fully connected layers with nonlinear activation functions to map input fea-

tures to class predictions. Each layer applies trainable weights and biases, followed by an activation function. We used ReLU, $f(z) = \max(0, z)$, in the hidden layers and softmax in the output layer to produce probabilities for the nine classes. The predicted class was the one with the highest probability.

Training minimised sparse categorical cross-entropy using integer class labels. Backpropagation computed the gradients, and Adam updated the weights in mini-batches [6, Chs. 6 and 8]. We also tested dropout after each hidden layer. Dropout randomly sets a proportion of activations to zero during training, which can reduce overfitting [6, Ch. 7].

Using the same extracted features for MLP and Random Forest allowed us to compare their learning methods under a common input representation. The comparison with CNN also involved a difference in representation, since CNN used image pixels rather than these feature vectors.

3.1.3 Convolutional Neural Network

A convolutional neural network (CNN) learns spatial features directly from images through local connections and shared weights [6]. A convolutional layer applies each learned filter across local image regions. Each filter combines values from all input channels and produces a feature map, followed here by a ReLU activation. Sharing the same weights across positions allows a local pattern to be detected in different parts of an image.

For a convolution with kernel size $k \times k$, C_{in} input channels and C_{out} output channels, the number of parameters, including biases, is

$$(k^2 C_{\text{in}} + 1) C_{\text{out}}.$$

Unlike a fully connected layer operating on flattened pixels, this parameter count does not depend on the spatial size of the input. However, computation still increases with the number of image positions.

Max pooling retains the largest activation within each local window, reducing spatial resolution and providing some tolerance to small shifts. Stacking convolutional layers allows later features to combine information from larger image regions. These properties are useful for tissue patches, where local textures and structures can occur at different positions.

Our CNN flattens the final feature maps and passes them through a fully connected hidden layer and a nine-class softmax output. The network is trained end-to-end using backpropagation and Adam to minimise sparse categorical cross-entropy. Dropout is applied during training, and validation-based early stopping selects the checkpoint in each hyperparameter trial.

3.2 Strengths and Weaknesses

Input representation and learning. RF and MLP both learn nonlinear relationships from the same 357-dimensional features. RF uses threshold-based splits and does not require feature standardisation. MLP learns successive feature transformations through gradient-based optimisation, making scaling, initialisation and learning rate important. Their compact input reduces computation, but both methods depend on the information retained by the fixed feature extractor.

CNN learns local filters directly from image pixels. Weight sharing allows the same pattern to be detected at different positions, which suits tissue patches without a single centred object [6]. It may distinguish local structures that block summaries lose. However, pooling also removes spatial detail, and a standard CNN does not guarantee invariance to rotations. Its suitability for images does not guarantee better performance on every dataset.

Model capacity and overfitting. RF reduces variance by averaging diverse trees [3], but deep trees can still fit details that do not generalise. Increasing the minimum leaf size or restricting depth can limit this flexibility. Adding trees mainly improves averaging, often with diminishing returns. For MLP and CNN, wider layers or more filters increase capacity and can increase overfitting risk. Dropout and validation-based early stopping provide ways to control this risk, although their benefit depends on the configuration and training budget.

Computational cost. RF training and memory requirements generally increase with the number and size of trees. The MLP operates on compact feature vectors, whereas CNN repeatedly applies filters across spatial positions. Convolution shares parameters, but this does not necessarily imply fewer total parameters or less computation than our feature-based MLP. The CNN's dense classifier also contributes substantially to its parameter count. Runtime therefore depends on the complete architecture, input representation, training duration and implementation.

Interpretability. RF can provide importance scores for named input features, making it easier to relate the model to colour and local statistics. These scores do not establish causal effects or fully explain individual predictions. MLP hidden units and CNN feature maps are less directly interpretable, although inspecting them can provide useful clues. None of these models is as straightforward to interpret as a single small decision tree.

3.3 Architecture and Hyperparameters

3.3.1 Random Forest Search Design

We used a staged validation search to limit computational cost. Table 2 lists the candidate settings. The search varied the number of features considered at each split, tree depth, minimum leaf size, and number of trees. Gini impurity and bootstrap sampling were retained throughout.

Table 2: RF hyperparameter settings explored across successive search stages, rather than as one complete Cartesian grid.

Hyperparameter	Candidate values	Rationale
`max_features`	`sqrt`, 0.1, 0.3	More candidate features may improve individual splits, but increase computation and similarity between trees.
`max_depth`	16, unlimited	Compare restricted tree complexity with deeper, more flexible partitions.
`min_samples_leaf`	1, 4	Larger leaves smooth predictions and may reduce overfitting.
`n_estimators`	100, 200, 400	Assess whether additional trees improve averaging enough to justify their cost.

Fractional `max_features` values specify the proportion of input features considered at each split. All search stages used the same 25,600 fitting examples and 6,400 validation examples.

First, six trials compared the three image representations with minimum leaf sizes of one and four. These trials used 100 trees, unlimited depth, `max_features=sqrt`, and seed 42. On the best representation, we then evaluated the 12 combinations of feature subset size, depth, and leaf size listed in Table 2, keeping 100 trees. Two results were reused from the representation stage, requiring 10 additional fits.

Next, we tested the two best structures with 200 and 400 trees, adding four fits. We then repeated each of the two leading configurations with seeds 17 and 73, adding another four fits. The reported RF search therefore comprised 24 training runs: six for representation comparison, ten additional runs for structural tuning, four for tree-count comparison, and four additional seed runs.

Final selection used mean validation macro F1 across seeds 42, 17, and 73, with mean accuracy breaking ties. These repetitions assessed sensitivity to forest randomness on one split, rather than variation across different splits. The selected configuration was refitted on all 32,000 training examples using seed 42.

3.3.2 MLP Architecture and Search Design

We used 357 input features, two ReLU hidden layers, and a nine-unit softmax output layer. Two hidden layers allowed successive nonlinear transformations without expanding the search to many network depths. Table 3 lists the three hyperparameters varied: hidden-layer widths, dropout rate, and Adam learning rate. Activation functions, optimiser type, and batch size were fixed, and early stopping was used in every trial.

Table 3: MLP hyperparameter search space and design rationale. Hidden-layer widths are listed in input-to-output order.

Hyperparameter	Candidate values	Rationale
Hidden-layer widths	$(128, 64)$; $(256, 128)$	Compare modelling capacity and computational cost, and assess whether a wider network improves validation performance.
Dropout rate	0.0; 0.2	Test whether moderate dropout improves generalisation relative to no dropout.
Learning rate	0.001; 0.0003	Compare optimisation behaviour and the performance–runtime trade-off under different update sizes.

Grid search evaluated all eight combinations on the same fitting and validation subsets. This small grid allowed us to compare all combinations within the available computational budget. Each model was trained from scratch for up to 60 epochs with a batch size of 128, resetting the random seed to 42 before each trial. Early stopping monitored validation cross-entropy with a patience of eight epochs and restored the weights with the lowest validation loss.

We ranked configurations by validation macro F1, using accuracy to break ties. Macro F1 gave each class equal weight. Thus, validation loss selected the checkpoint within each run, while macro F1 selected the configuration across runs.

The selected configuration was then trained from scratch on all 32,000 training examples, using the epoch count chosen during validation. Within the MLP experiments, test data were not used to fit the scaler, select hyperparameters, or determine the training epoch count.

3.3.3 CNN Architecture and Search Design

We used a CNN with three convolutional blocks and standardised $28 \times 28 \times 3$ images as input. Each block consisted of a 3×3 convolution with stride one, same padding and ReLU activation, followed by 2×2 max pooling and dropout. The

blocks used f, $2f$ and $4f$ filters, where f was the base filter count. Pooling reduced the spatial dimensions from 28×28 to 14×14, 7×7 and finally 3×3.

The final feature maps were flattened and passed through a 128-unit ReLU layer, dropout and a nine-unit softmax output layer. We trained the networks using Adam and sparse categorical cross-entropy with integer class labels. The networks had 98,601 trainable parameters for $f = 16$ and 241,993 for $f = 32$.

We chose small kernels to capture local patterns with relatively few weights per filter. Pooling reduced the number of inputs to the dense layer, while increasing the filter count in later blocks allowed more feature maps to be learned. We kept the number of blocks, kernel size and dense-layer width fixed, and focused the search on filter count, dropout and learning rate.

Table 4 lists the three hyperparameters varied. The same dropout rate is applied after every convolutional block and after the hidden dense layer.

Table 4: CNN hyperparameter search space and design rationale. All eight combinations were evaluated.

Hyperparameter	Candidate values	Rationale
Base filters f	16; 32	Compare feature-learning capacity and cost. More filters may improve the fit, but increase computation and the risk of overfitting.
Dropout rate	0.0; 0.3	Test whether moderate regularisation improves validation performance relative to no dropout. It may also slow learning.
Learning rate	0.001; 0.0003	Compare update sizes within a fixed epoch budget. Smaller updates may improve stability but require more epochs to reach a useful solution.

We evaluated all eight hyperparameter combinations on the shared fitting and validation subsets. This small grid allowed us to compare settings while holding the other hyperparameters fixed. We used one validation split to limit computation; sensitivity to different splits was not assessed.

Each configuration was trained from scratch with seed 42, batch size 128 and a maximum of 30 epochs. Early stopping monitored validation cross-entropy with a patience of five epochs and restored the weights with the lowest validation loss. We then ranked the configurations by validation macro F1, using accuracy to break ties. As with the MLP, validation loss determined the checkpoint within each run, while macro F1 determined which configuration was selected.

For each run, we recorded validation scores, the best epoch, the number of epochs completed and fitting time. Fitting time included training and validation within each epoch, but excluded preprocessing, model construction and the separate prediction step used to calculate validation macro F1.

We retrained the selected configuration from scratch on all 32,000 training images for the number of epochs chosen during validation. The notebook specifies both the final configuration and epoch count explicitly, so final training can be run without repeating the search.

4 Results and Discussion

4.1 Hyperparameter Tuning

4.1.1 Random Forest Tuning Results

Effect of image representation. Table 5 compares the three representations while holding the tree count, depth, feature-sampling rule, and random seed fixed. The 357-dimensional descriptor achieved the highest validation macro F1 at both minimum leaf sizes.

Table 5: RF validation macro F1 for three image representations, using 100 trees, unlimited depth, `max_features=sqrt`, and seed 42. Leaf size refers to `min_samples_leaf`.

Representation	Dimensions	Leaf size 1	Leaf size 4
Raw RGB	2,352	0.6024	0.6052
Pooled RGB	588	0.6230	0.6142
Colour and local texture	357	**0.8252**	**0.8222**

With a minimum leaf size of one, pooling increased validation macro F1 from 0.6024 to 0.6230. The colour and local texture descriptor achieved 0.8252 and also performed best with a minimum leaf size of four. We therefore used this 357-dimensional representation for the remaining RF experiments. These results suggest that the local summaries and colour statistics provided more useful inputs for this forest than the raw or pooled pixel values.

The comparison used the same RF parameter settings, but it did not isolate the contribution of each descriptor component. In particular, `max_features=sqrt` selects different numbers of candidate features for representations with different dimensions. The results apply to the RF configurations tested here; they do not establish which input representation would work best for a CNN.

Effect of structural hyperparameters. Using the 357-dimensional descriptor, increasing `max_features` improved validation macro F1 across all four depth and leaf-size combinations in Figure 2. This improvement came with a substantial increase in fitting time.

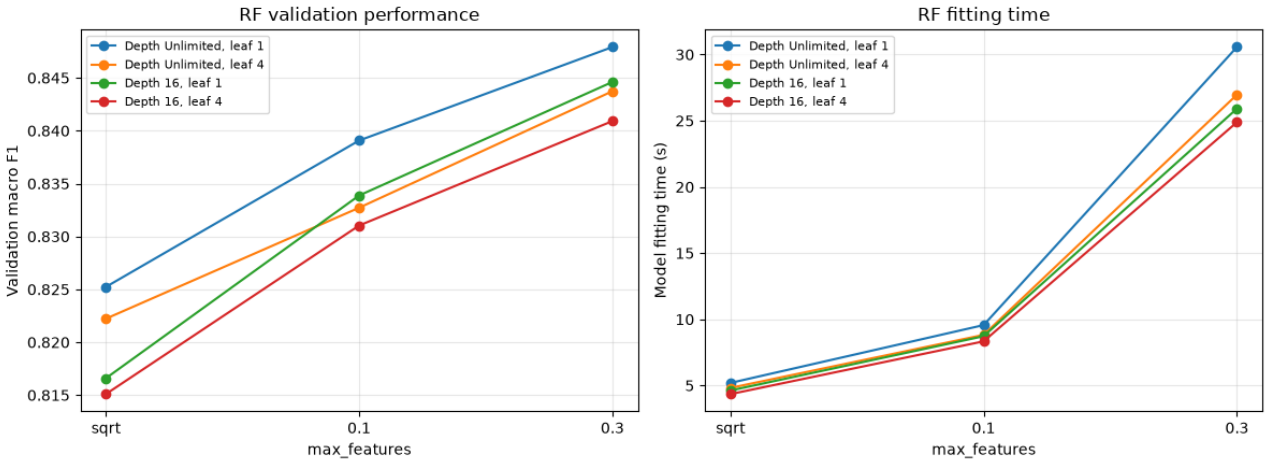


Figure 2: RF structural tuning on the 357-dimensional representation. Left: validation macro F1. Right: fitting time. All configurations used 100 trees, the same fitting and validation subsets, and seed 42.

For unlimited depth and a minimum leaf size of one, changing `max_features` from `sqrt` to 0.3 increased validation macro F1 from 0.8252 to 0.8479, while fitting time increased from 5.19 to 30.56 seconds. These settings consider 18 and 107 candidate features per split, respectively. Evaluating more candidate features may help the trees find better splits, but it also requires more computation.

At `max_features=0.3`, limiting the maximum depth to 16 reduced validation macro F1 from 0.8479 to 0.8446. Keeping unlimited depth and increasing the minimum leaf size from one to four gave 0.8437. Neither restriction improved validation performance in this comparison. We therefore retained unlimited depth and a minimum leaf size of one for the subsequent tree-count experiments, alongside the depth-16 configuration.

Tree count and runtime. The two leading structures were evaluated with 200 and 400 trees (Table 6). For unlimited depth, increasing the tree count from 200 to 400 raised validation macro F1 from 0.8476 to 0.8495, while fitting time approximately doubled from 60.68 to 121.40 seconds. With maximum depth limited to 16, macro F1 increased from 0.8426 to 0.8445. In both cases, the additional trees produced a small improvement at roughly twice the fitting cost. The unlimited-depth configurations performed better at both tree counts and were taken forward to the seed-stability comparison.

Table 6: RF tree-count experiments using the 357-dimensional descriptor, `max_features=0.3`, `min_samples_leaf=1`, and seed 42.

Maximum depth	Trees	Val. accuracy	Val. macro F1	Fit time (s)
Unlimited	200	0.8525	0.8476	60.68
Unlimited	400	0.8541	0.8495	121.40
16	200	0.8478	0.8426	52.05
16	400	0.8495	0.8445	105.99

Stability across random seeds. The unlimited-depth forests with 200 and 400 trees were repeated using seeds 17 and 73, in addition to seed 42. Table 7 summarises the three runs for each candidate on the same validation split.

Table 7: RF validation results across seeds 42, 17, and 73. SD denotes the sample standard deviation. Six decimal places are retained to show the small differences.

Trees	Mean val. accuracy	Mean val. macro F1	SD of macro F1
200	0.853177	0.848502	0.001855
400	0.853542	0.848946	0.001776

The 400-tree forest achieved a slightly higher mean validation macro F1 and was selected according to our selection rule. Its advantage over 200 trees was approximately 0.00044. This difference was smaller than the seed-to-seed standard deviation for either configuration, so these three runs do not establish a clear performance advantage. The 200-tree forest remained a reasonable alternative when fitting time was important. Because all runs used the same validation split, the standard deviations describe variation due to model randomness, not variation across different data splits.

The final RF configuration used the 357-dimensional colour and local texture descriptor, 400 trees, unlimited depth, `max_features=0.3`, and `min_samples_leaf=1`.

4.1.2 MLP Tuning Results

Table 8 reports the results of all eight configurations on the shared validation set. The selected configuration used hidden-layer widths of $(256, 128)$, a dropout rate of 0.2, and a learning rate of 0.0003. It achieved the highest validation macro F1 of 0.8606 and the highest validation accuracy of 0.8661.

Table 8: Validation results for all eight MLP configurations. Bold values identify the selected configuration. Best epoch is determined by minimum validation loss; epochs run includes the early-stopping patience period. Time includes model construction and fitting, including validation during training, but excludes feature scaling and post-training prediction.

Trial	Hidden widths	Dropout	Learning rate	Best epoch	Epochs run	Val. accuracy	Val. macro F1	Time (s)
1	$(128, 64)$	0.0	0.0010	7	15	0.8344	0.8311	2.25
2	$(128, 64)$	0.0	0.0003	21	29	0.8419	0.8380	3.82
3	$(256, 128)$	0.0	0.0010	8	16	0.8348	0.8322	3.02
4	$(256, 128)$	0.0	0.0003	13	21	0.8455	0.8407	3.81
5	$(128, 64)$	0.2	0.0010	21	29	0.8598	0.8547	4.08
6	$(128, 64)$	0.2	0.0003	50	58	0.8620	0.8570	8.71
7	$(256, 128)$	0.2	0.0010	13	21	0.8622	0.8577	4.24
8	$(\mathbf{256}, \mathbf{128})$	**0.2**	**0.0003**	**33**	**41**	**0.8661**	**0.8606**	**7.68**

Figure 3 visualises the matched comparisons in Table 8. Each pair of bars compares the two dropout rates while keeping the hidden-layer widths and learning rate fixed.

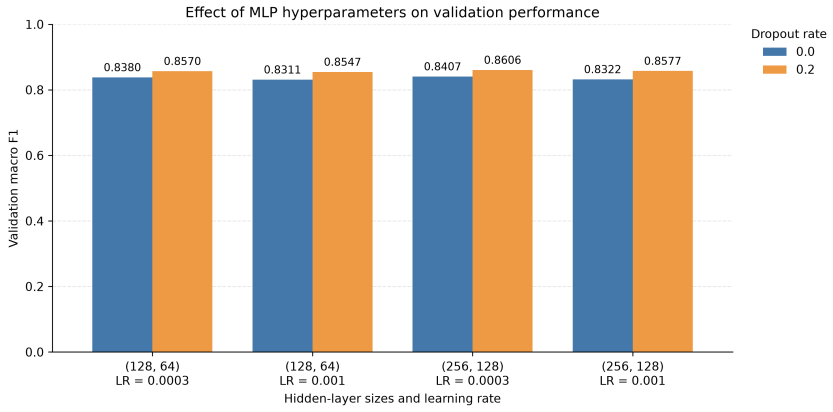


Figure 3: Validation macro F1 across the eight MLP configurations. Each group represents one combination of hidden-layer widths and learning rate; colours distinguish dropout rates of 0.0 and 0.2. Values above the bars show macro F1 to four decimal places.

Effect of dropout. Figure 3 shows that dropout of 0.2 improved validation macro F1 in all four matched comparisons, with gains of 0.0190–0.0255. For example, the wider network at a learning rate of 0.001 improved from 0.8322 to 0.8577. This was the largest effect among the settings tested and supports the use of regularisation for this model. It does not show that 0.2 is the best dropout rate beyond the two values tested.

Effect of network width. Increasing the hidden-layer widths from $(128, 64)$ to $(256, 128)$ improved macro F1 by only 0.0011–0.0036. Although the wider model performed better in all four comparisons, increasing capacity had a smaller effect than adding dropout.

Effect of learning rate and runtime. The lower learning rate improved validation macro F1 in all four matched comparisons, but required more epochs and longer training. For the wider model with dropout, reducing the learning rate from 0.001 to 0.0003 raised macro F1 from 0.8577 to 0.8606, while runtime increased from 4.24 to 7.68 seconds. The improvement was small relative to the additional fitting time. We selected this configuration because validation macro F1 was our primary selection criterion.

These trends were consistent with our expectations for regularisation and smaller learning steps. However, one seed and one validation split are insufficient to establish whether the small differences would persist across repeated runs.

Learning dynamics and early stopping. In Figure 4, both losses fell rapidly at first. Later, training loss kept decreasing while validation loss stayed near 0.40. Validation accuracy also levelled off as training accuracy continued to rise. This suggests that further training mainly improved the fit to the training data rather than validation performance.

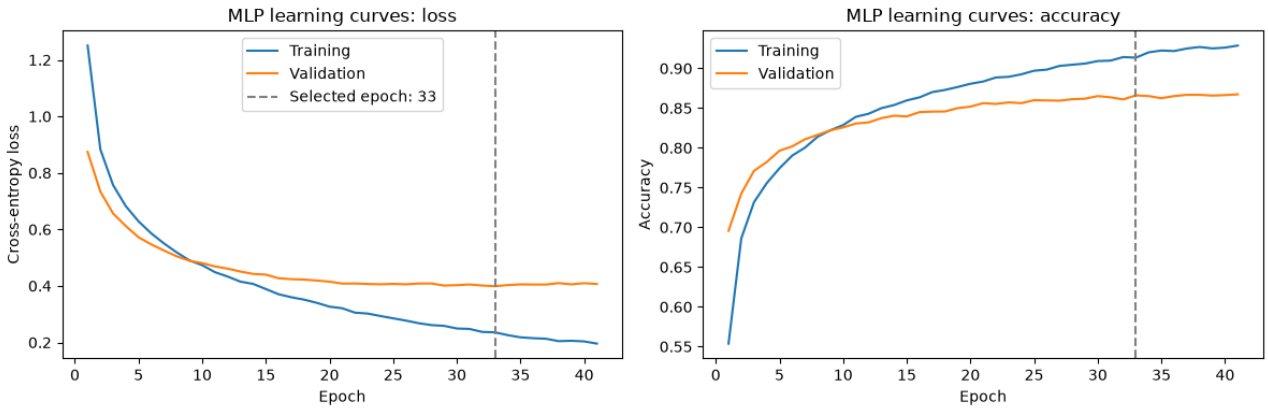


Figure 4: Training and validation learning curves for the selected MLP configuration: hidden layers of 256 and 128 units, dropout rate 0.2, and learning rate 0.0003. The dashed line marks epoch 33, which achieved the lowest validation loss.

Validation loss was lowest at epoch 33, and training stopped at epoch 41 after eight epochs without improvement. The epoch-33 weights were restored, and 33 epochs were subsequently used for final retraining.

The initial advantage of validation over training may partly reflect dropout being disabled during validation and training metrics being collected while the weights were being updated. The curves therefore reflect different evaluation conditions as well as differences between the data subsets.

4.1.3 CNN Tuning Results

Table 9 reports all eight CNN configurations. Trial 7 achieved the highest validation macro F1 of 0.9354 and accuracy of 0.9367. It used 32 base filters, dropout of 0.3, and a learning rate of 0.001.

Table 9: Validation results for all eight CNN configurations. Bold values identify the selected configuration. Best epoch is determined by minimum validation loss, and validation metrics are evaluated after restoring the corresponding weights. Time covers model fitting, including validation during training, but excludes image preprocessing, model construction and post-training prediction.

Trial	Base filters	Dropout	Learning rate	Best epoch	Epochs run	Val. accuracy	Val. macro F1	Time (s)
1	16	0.0	0.0010	14	19	0.8667	0.8637	30.65
2	16	0.0	0.0003	30	30	0.8516	0.8477	46.34
3	16	0.3	0.0010	28	30	0.8955	0.8937	48.91
4	16	0.3	0.0003	28	30	0.8309	0.8278	48.93
5	32	0.0	0.0010	9	14	0.8722	0.8685	50.34
6	32	0.0	0.0003	13	18	0.8381	0.8339	65.16
7	**32**	**0.3**	**0.0010**	**29**	**30**	**0.9367**	**0.9354**	**112.48**
8	32	0.3	0.0003	28	30	0.8939	0.8927	113.37

Effect of dropout. Dropout of 0.3 improved validation macro F1 in three of the four matched comparisons. For 32 base filters and learning rate 0.001, macro F1 increased from 0.8685 to 0.9354. However, with 16 base filters and learning rate 0.0003, it decreased from 0.8477 to 0.8278. Dropout therefore helped most configurations, but its effect depended on the other settings. This partly matched our expectation that dropout would help control overfitting. Its gains were larger for the 32-filter networks at both learning rates, which is consistent with stronger regularisation being useful for the higher-capacity models. However, dropout also makes optimisation more difficult by randomly removing activations during training. For the 16-filter network at learning rate 0.0003, the combination of lower capacity, smaller updates and dropout may have limited learning within 30 epochs. This offers a possible explanation for the lower score, rather than evidence that dropout is generally unsuitable for this architecture. Training curves and repeated runs would be needed to distinguish slower optimisation from other sources of variation.

Effect of filter count and runtime. With dropout enabled, increasing the base filters from 16 to 32 improved macro F1 from 0.8937 to 0.9354 at learning rate 0.001, and from 0.8278 to 0.8927 at learning rate 0.0003. Without dropout, the changes were smaller and not consistently positive. The larger network also required more time: trials 3 and 7 both ran for 30 epochs, but fitting time increased from 48.91 to 112.48 seconds. The benefit of additional filters therefore came at a higher computational cost.

Effect of learning rate. Learning rate 0.001 outperformed 0.0003 in all four matched comparisons. This differs from the MLP results, where the lower learning rate performed better. However, three of the four CNN configurations using 0.0003 reached the 30-epoch limit, with their lowest validation losses at epochs 28–30. A longer training budget could change their ranking. These results support 0.001 within the tested budget, rather than establishing it as the best learning rate in general.

All comparisons used one validation split and one random seed. They describe the tested configurations and do not establish that the same differences would persist across other splits or initialisations.

Learning curves and epoch selection. Figure 5 shows the validation losses of all eight configurations and the learning curves of the selected CNN. Trial 7 achieved its lowest validation loss at epoch 29 and ran to the 30-epoch limit; it was not stopped early by the patience criterion. The epoch-29 weights were restored for validation evaluation, and 29 epochs were used for final retraining.

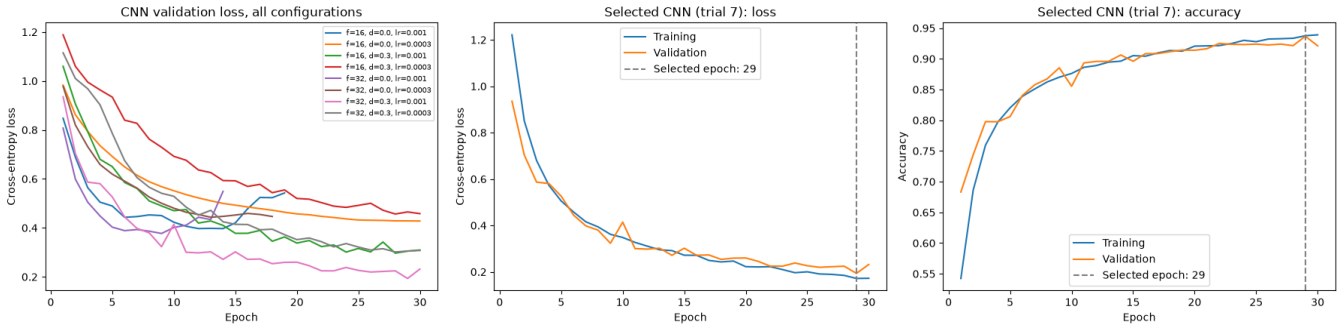


Figure 5: CNN learning curves. Left: validation loss for all eight configurations. Centre and right: training and validation loss and accuracy for the selected configuration (32 base filters, dropout 0.3, learning rate 0.001). The dashed lines mark epoch 29, which achieved the lowest validation loss in the selected trial. The selected trial ran for 30 epochs.

For the selected CNN, training and validation loss generally decreased together, with the lowest validation loss at epoch 29 and a rise at epoch 30. This supported using the epoch-29 checkpoint rather than the final epoch. The small gap between the curves should be interpreted with care: training metrics were recorded with dropout enabled and changing weights, whereas validation used fixed weights with dropout disabled. These curves alone cannot rule out overfitting.

4.2 Final Model Comparison

Table 10 summarises the selected RF, MLP and CNN models. Each model was refitted on all 32,000 training examples and evaluated on the same 8,000 test examples. Hyperparameters and neural-network training durations were selected using validation results.

Table 10: Selected configurations and final test performance. Times refer to the final training and test evaluation runs, excluding hyperparameter search. Measurement boundaries are specified below.

Model	Selected configuration	Test accuracy	Macro F1	Weighted F1	Training time (s)	Test time (s)
RF	400 trees; unlimited depth; `max_features`=0.3; `min_samples_leaf`=1	0.8608	0.8565	0.8597	163.31	0.08
MLP	Hidden widths $(256, 128)$; dropout 0.2; learning rate 0.0003; 33 training epochs	0.8772	0.8737	0.8770	6.59	0.09
CNN	Convolutional filters $(32, 64, 128)$; dense width 128; dropout 0.3; learning rate 0.001; 29 training epochs	**0.9319**	**0.9303**	**0.9316**	127.74	0.41

Timing note. RF times cover model fitting and prediction only. MLP training time includes standardisation, model construction and fitting; its test time includes standardisation and prediction. RF and MLP times exclude feature extraction, which is measured separately in the notebook. CNN training time includes image preprocessing, model construction and fitting; its test time includes preprocessing and prediction. These measurements describe the implemented pipelines and are not a controlled comparison of algorithm speed.

The CNN achieved the highest scores on all three test metrics. Its accuracy exceeded the MLP's by approximately 5.46 percentage points and the RF's by 7.11 percentage points. Its macro F1 was also higher, showing that the improvement was not limited to the more frequent classes.

The selected MLP achieved test accuracy 1.65 percentage points higher than the selected RF. Both used the same 357-dimensional descriptor, with additional standardisation for the MLP. The MLP's learned combinations of these features may have captured relationships that were less effectively represented by the RF's individual tree splits. The CNN's advantage agrees with our expectation that learned spatial filters would be useful for distinguishing tissue textures and structures. Although the fixed descriptor retains coarse spatial information, its summary statistics discard some local detail that remains available to the CNN. However, these results compare our complete pipelines: differences in preprocessing, search spaces and training procedures mean that the performance gaps cannot be attributed to model architecture alone.

The MLP had the shortest recorded final training time (6.59 s), compared with 127.74 s for the CNN and 163.31 s for the RF. Its compact feature input and two hidden layers help explain its lower training cost compared with the CNN, which repeatedly applies convolutions to image feature maps. The RF fitted 400 trees with no maximum depth, which helps explain its substantial training cost despite being a non-neural model. These timings also depend on the implementation, hardware and measurement boundaries described in the timing note below the results table. In particular, feature extraction was excluded from the RF and MLP training times. The MLP therefore offered a useful accuracy–training-time trade-off in this run, while the CNN provided higher predictive performance at greater training cost than the MLP. This is a comparison of the selected implementations, rather than a general speed ranking of the three algorithms.

Class 7 had the lowest F1 score for all three models. Although the CNN improved its F1 to 0.8223, this class remained more difficult than the others. The class-level analysis below examines the remaining errors.

4.3 Analysis of Model Performance

4.3.1 Random Forest Test Performance and Error Analysis

The selected RF misclassified 1,114 of the 8,000 test images. Figure 6 shows how these errors were distributed across classes.

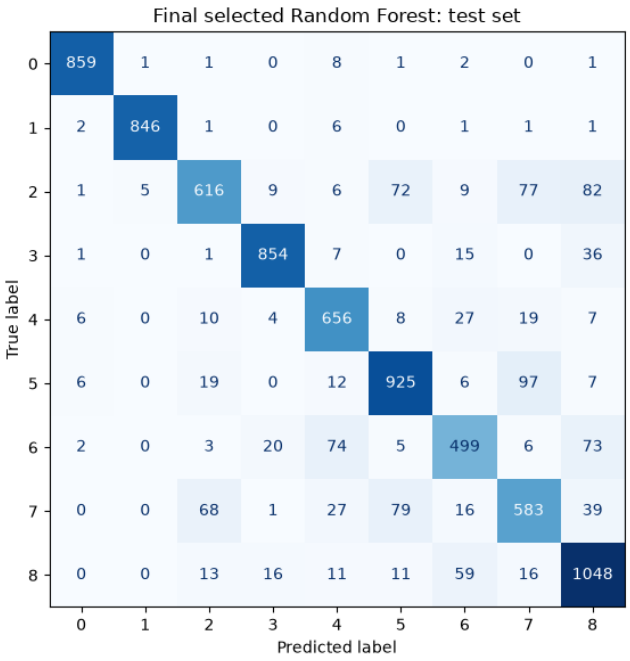


Figure 6: Confusion matrix of the selected Random Forest on the 8,000 test images. Rows indicate true classes and columns indicate predicted classes. Values are sample counts.

Figure 6 shows substantial differences between classes. Classes 0 and 1 had F1 scores of 0.9817 and 0.9895, respectively. Class 7 had the lowest F1 score (0.7233), followed by classes 6 (0.7584) and 2 (0.7657).

The largest individual confusion was class 5 being predicted as class 7 (97 images), while 79 class-7 images were predicted as class 5. Class 2 was frequently predicted as classes 8 and 7, with 82 and 77 errors, respectively. Class 6 was also confused with classes 4 and 8, with 74 and 73 errors. These errors show which classes the selected forest struggled to distinguish using the fixed descriptors. The confusion counts alone do not establish which visual characteristics caused the mistakes.

Final model fitting took 163.31 seconds. Training feature extraction took a further 2.63 seconds, while test feature extraction and prediction took 0.63 and 0.08 seconds, respectively. Prediction was fast once the features were available, but feature extraction accounted for most of the measured test processing time. These timings exclude hyperparameter search.

4.3.2 MLP Test Performance and Error Analysis

The MLP achieved test macro F1 of 0.8737, compared with 0.8606 on validation data. The evaluation subsets and training sizes differed, so this increase cannot be attributed to retraining alone.

Variation across classes. Classes 0, 1, and 3 achieved F1 scores of approximately 0.96 or higher, while classes 2, 6, and 7 scored below 0.80. Class 7 had the lowest F1 of 0.7543, with precision of 0.7337 and recall of 0.7761. Class 2 also had low recall of 0.7412, meaning that roughly one quarter of its examples were misclassified. These differences show that the similar macro and weighted F1 scores did not imply equally strong performance across classes.

Patterns of misclassification. Figure 7 shows the error counts and row-normalised proportions. The latter allows comparison across classes with different test-set sizes.

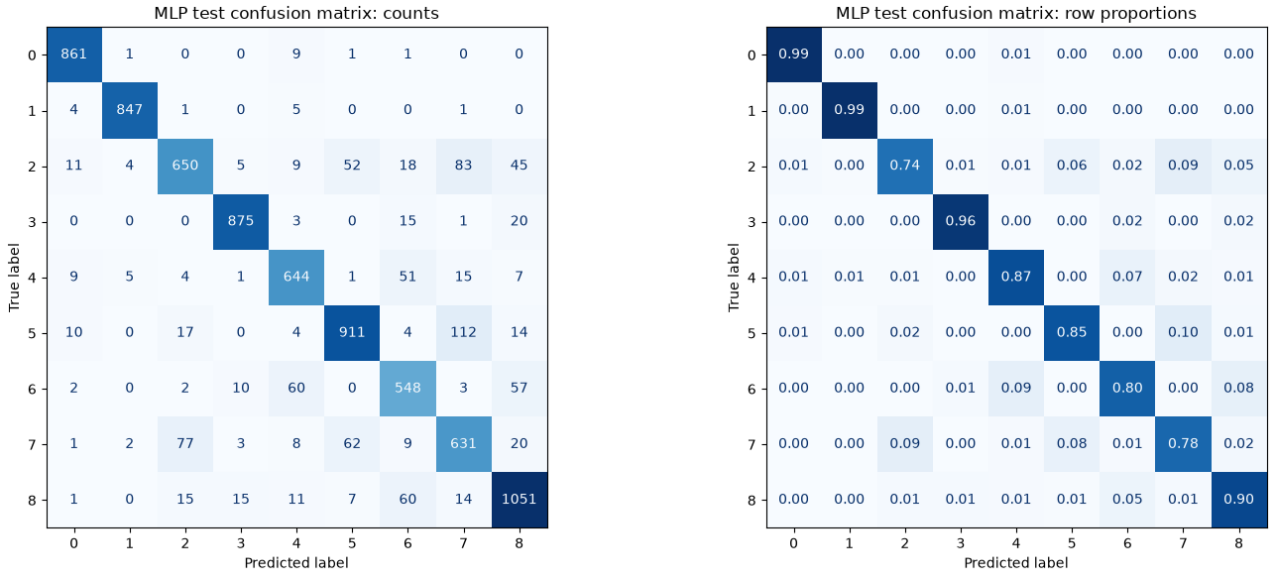


Figure 7: Confusion matrices of the selected MLP on the 8,000 test images. Left: sample counts. Right: proportions normalised within each true class. Rows indicate true classes and columns indicate predicted classes.

The largest error count was from class 5 to class 7: 112 examples, or 10.45% of class 5. Another 62 errors occurred in the reverse direction. Classes 2 and 7 were also confused with each other, with 83 and 77 errors, respectively. Predictions from classes 5 and 2 into class 7 help explain its low precision. Class 6 was frequently assigned to classes 4 and 8, with 60 and 57 errors.

The main difficulty was distinguishing cancer-associated stroma (class 7) from smooth muscle (class 5) and debris (class 2). Errors occurred in both directions, so improving class 7 recall alone would not resolve these confusions. A useful next step would be to inspect examples from these pairs to assess whether the fixed colour and texture features miss details needed to separate them.

Computational cost. Final standardisation, model construction and fitting took 6.59 seconds; test standardisation and prediction took 0.09 seconds. Training feature extraction took a further 2.81 seconds, and test feature extraction took 0.63 seconds. Feature extraction therefore accounted for most of the measured test processing time, although prediction on the precomputed features was fast. These measurements exclude file loading, metric calculation and hyperparameter search.

4.3.3 CNN Test Performance and Error Analysis

The selected CNN misclassified 545 of the 8,000 test images. Its test macro F1 of 0.9303 was slightly below the validation score of 0.9354. However, the evaluation subsets and training sizes differed, so this difference alone does not measure overfitting.

Variation across classes. Classes 0 and 1 achieved F1 scores of 0.9886 and 0.9924, respectively. Class 7 (cancer-associated stroma) had the lowest F1 score of 0.8223, followed by class 2 (debris) at 0.8586. Class 7 had precision of 0.9041 but recall of 0.7540: predictions assigned to this class were usually correct, but approximately one quarter of its true examples were missed. Class 2 showed the opposite pattern, with high recall of 0.9384 but lower precision of 0.7913.



Figure 8: Confusion matrices of the selected CNN on the 8,000 test images. Left: sample counts. Right: proportions normalised within each true class. Rows indicate true classes and columns indicate predicted classes.

Patterns of misclassification. Figure 8 shows that the largest confusion was class 7 being predicted as class 2 (126 images), followed by class 7 being predicted as class 5 (61 images). Together, these accounted for 187 of the 200 errors involving true class 7. Only 20 class-2 images were predicted as class 7, showing that this confusion was strongly asymmetric.

Other frequent errors included class 8 being predicted as class 2 (40 images) and class 5 being predicted as class 7 (33 images). The eight most frequent confusion pairs accounted for 63.7% of all errors. These counts identify the remaining weaknesses, but do not by themselves explain their visual causes.

Prediction confidence. The mean maximum softmax probability was 0.958 for correct predictions and 0.712 for incorrect predictions. However, 20.0% of errors had a maximum probability above 0.9. High model confidence therefore did not guarantee a correct prediction. These values describe the model's outputs; we did not establish that they were calibrated probabilities of correctness.

Computational cost. Final image preprocessing, model construction and fitting took 127.74 seconds. Test preprocess-

ing and prediction took 0.41 seconds. These measurements exclude hyperparameter search, file loading and metric calculation.

4.3.4 Class-Level Comparison

Table 11 compares the three models on each test class. The CNN achieved the highest F1 in all nine classes, while the MLP outperformed the RF in seven classes.

Table 11: Per-class F1 scores on the same 8,000 test images. Bold values indicate the highest score for each class. Support is the number of test images in that class.

Class	Description	Support	RF	MLP	CNN
0	Adipose	873	0.9817	0.9718	**0.9886**
1	Background	858	0.9895	0.9866	**0.9924**
2	Debris	877	0.7657	0.7912	**0.8586**
3	Lymphocytes	914	0.9395	0.9600	**0.9768**
4	Mucus	737	0.8497	0.8644	**0.9372**
5	Smooth muscle	1,072	0.8514	0.8651	**0.9310**
6	Normal colon mucosa	682	0.7584	0.7896	**0.9256**
7	Cancer-associated stroma	813	0.7233	0.7543	**0.8223**
8	Colorectal adenocarcinoma epithelium	1,174	0.8493	0.8802	**0.9405**

The largest CNN improvement over the MLP was for class 6, where F1 increased from 0.7896 to 0.9256. Improvements were also substantial for classes 4 and 7. In contrast, all three models already performed well on classes 0 and 1, leaving less room for improvement.

Class 7 remained the weakest class for every model. Its F1 increased from 0.7233 with RF to 0.7543 with MLP and 0.8223 with CNN, but the CNN's errors were concentrated in predictions of class 7 as class 2. The higher overall scores therefore did not eliminate all class-specific weaknesses.

The MLP's higher aggregate scores also did not imply an improvement in every class: RF retained slightly higher F1 for classes 0 and 1. This illustrates why class-level results are useful alongside accuracy and macro F1.

5 Conclusion

Main findings. The CNN achieved the highest test accuracy (0.9319) and macro F1 (0.9303), followed by the MLP (0.8772 and 0.8737) and RF (0.8608 and 0.8565). The CNN also achieved the highest F1 in every class, although class 7 remained the weakest class for all three models.

Feature representation had a large effect on RF performance. For the MLP, dropout improved all four matched configurations, while increasing network width gave smaller gains. CNN results showed that the effects of dropout and filter count depended on the other settings. The MLP offered a useful balance between predictive performance and recorded training time, while the CNN achieved better scores with a higher training cost than the MLP. Interpretability remains a limitation of this comparison. The RF and MLP used colour and texture features whose definitions are explicit, but this does not explain how each model reached an individual prediction. The CNN learned its features directly from images, and we did not analyse which image regions influenced its decisions. Its higher predictive performance therefore does not, by itself, make its predictions easier to explain.

Limitations. All models were selected using one fixed validation split and limited search spaces. RF was compared across three seeds, but MLP and CNN used one seed. The RF repeats measured model randomness on the same split, not uncertainty across different data partitions. The CNN search was also limited to 30 epochs, which may have disadvantaged some slower-learning configurations.

The models differed in input representation, preprocessing and search budgets, so their results do not isolate the effect of architecture alone. Runtime measurements also covered different processing steps. Patient-level independence could not be verified from the supplied arrays without patient identifiers.

Preliminary RF and CNN experiments had already inspected the test set before the final evaluation. The results reported here therefore do not represent a first evaluation on a completely untouched test set. The final configurations were selected using validation results, but earlier test exposure remains a limitation.

Future work. Repeated validation splits and additional random seeds would help assess whether the observed differences persist. Patient-grouped evaluation would provide a stronger check of generalisation if patient identifiers became available. Feature ablation could clarify which parts of the fixed descriptor contribute most, while inspection of confused class pairs could guide further experiments.

For the CNN, a longer training budget and suitable image augmentation could be evaluated using training and validation data. Consistent timing boundaries would also make computational comparisons more informative. Further development should use a fresh held-out test set for final evaluation where possible.

6 Reflection

Member A: 550394172

The most important thing I learned from the Random Forest experiments was that the input representation can matter more than increasing model complexity. With 100 trees, unlimited depth and the same feature-sampling rule, replacing raw RGB pixels with colour and local statistical features increased validation macro F1 from 0.6024 to 0.8252. This helped me understand why an algorithm needs features that make useful patterns accessible to its learning mechanism. It also taught me to interpret comparisons carefully: the representations differed in dimensionality, so the `sqrt` rule considered different numbers of features per split. The experiment supported the compact representation for this pipeline, but did not isolate the contribution of every feature component.

I also learned to distinguish a higher recorded score from a convincing practical improvement. Increasing the forest from 200 to 400 trees approximately doubled fitting time, while the mean validation macro F1 across three seeds improved by only about 0.00044, less than the seed-to-seed standard deviations. Although 400 trees won under our stated selection rule, 200 trees remained a reasonable choice when computation mattered. In future work, I would repeat promising settings across validation splits and report uncertainty alongside runtime before recommending a more expensive model. This experience made me more careful about both model selection and the strength of the conclusions I draw.

Member B: 550378183

The main lesson I learned was to judge model complexity using validation results rather than training performance. Increasing network width gave only small gains, whereas dropout had a larger effect. The learning curves also showed that training loss could keep falling without a comparable improvement on validation data. This made the purpose of regularisation and early stopping clearer to me.

Examining individual classes changed how I interpreted the overall result. Accuracy of 0.8772 looked encouraging, but class 7 had an F1 score of only 0.7543. In future experiments, I would examine class-level errors alongside aggregate scores and repeat promising settings across seeds before treating a small improvement as reliable.

Member C: 550894694

The most important thing I learned is to diagnose training curves and preprocessing before expanding a hyperparameter search. My initial CNN achieved approximately 0.80 test accuracy as a baseline, but the training and validation curves provided the evidence for debugging: both accuracies rose slowly with little separation. This suggested underfitting or slow optimisation rather than a clear overfitting problem. I therefore investigated the input pipeline before assuming that a larger architecture or stronger regularisation was needed. The initial inputs were scaled to $[0, 1]$ but were not standardised. Channel standardisation can improve the conditioning of optimisation, although scaling alone does not necessarily prevent a CNN from learning. During these preliminary experiments, validation macro F1 increased from approximately 0.82 to 0.93 after introducing channel standardisation. This was a substantial improvement in our experiments, but without a controlled ablation I cannot attribute the entire difference to standardisation alone. The experience made the lecture material on normalisation and optimisation more concrete for me.

I also learned to verify each step of the pipeline rather than trust it. Checking the mean of the standardised data revealed a numerical discrepancy when statistics were accumulated in 32-bit floating point over millions of pixel values. This motivated computing channel statistics in 64-bit precision and checking the transformed data explicitly. I had also expected batch normalisation to help, but observed different behaviour during training and inference on unstandardised inputs. These experiences showed me the value of evidence-driven debugging. They also taught me to describe evaluation procedures precisely: the preliminary CNN had been evaluated on the test set as a baseline, while subsequent hyperparameter selection used validation results. Baseline and final test scores were compared only after fixing the final configurations, and that comparison was not used to revise the models.

7 AI Acknowledgement

We used ChatGPT, OpenAI Codex and Claude Code to support this assignment.

For the Random Forest component, OpenAI Codex assisted with code development, experiment execution and summaries of experiment records. For the MLP component and notebook integration, ChatGPT assisted with environment troubleshooting, code review, interpretation of outputs and combining the members' contributions. For the CNN component, Claude Code assisted with code development, execution, debugging and drafting explanatory text.

ChatGPT also assisted with combining the report sections, LaTeX formatting, and checking reported values against the saved notebook outputs. The numerical results came from experiments executed locally, rather than from AI-generated estimates. OpenAI Codex also assisted with drafting the RF reflection from the experiment records and editing the evaluation description and CNN reflection.

References

[1] J. Yang et al., "MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Scientific Data*, vol. 10, article 41, 2023. doi:10.1038/s41597-022-01721-8.

[2] I. Ullah, "Ensemble Methods," COMP5318/COMP4318, Semester 2, Week 5B, University of Sydney, 2026. Lecture slides prepared by I. Koprinska.

[3] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. doi:10.1023/A:1010933404324.

[4] Scikit-learn developers, "RandomForestClassifier," *Scikit-learn documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html.

[5] University of Sydney, "COMP5318/COMP4318 Assignment 2: Image Classification," Semester 2, 2026. Assignment specification.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.